Contents

ClearWater-Riverine User's Manual

Version 1.0
U.S. Army Engineer Research and Development Center (ERDC)
Environmental Laboratory (EL)

Table of Contents

---

1. Introduction

ClearWater-Riverine is a two-dimensional (2D) water quality transport model designed to simulate the advection and diffusion of conservative constituents in complex river systems and floodplains. Developed by the U.S. Army Engineer Research and Development Center (ERDC), Environmental Laboratory (EL), this modern Python-based tool provides researchers and engineers with a powerful platform for water quality modeling.

1.1 Purpose and Scope

This user's manual provides comprehensive guidance for using ClearWater-Riverine, from initial installation through advanced modeling applications. The manual is designed for:

- Water quality modelers
- Environmental engineers
- Researchers studying riverine systems
- Students learning water quality modeling

1.2 What's New in Version 1.0

- Modern Python implementation using NumPy, SciPy, and Pandas
- Integration with Jupyter Notebooks for interactive modeling
- Coupling capabilities with Temperature Simulation Module (TSM) and Nutrient Simulation Module (NSM)
- Support for unstructured grids from HEC-RAS 2D models
- Efficient sparse matrix operations for large-scale simulations

1.3 System Requirements

- Python 3.10 or higher
- Miniconda or Anaconda Distribution
- Minimum 8 GB RAM (16 GB recommended for large models)
- 10 GB available disk space

---

2. Overview

2.1 Model Description

ClearWater-Riverine simulates the transport of heat and water quality constituents in riverine systems through:

- **Advection**: Transport of constituents with water flow
- **Diffusion**: Mixing and spreading due to turbulence and dispersion
- **Mass Conservation**: Ensuring proper mass balance throughout the domain

The model assumes vertical homogeneity, making it ideal for riverine systems where longitudinal and lateral changes dominate over vertical stratification.

2.2 Key Features

2.2.1 Computational Features

- Unstructured grid support for complex geometries
- Sparse matrix operations for computational efficiency
- Adaptive time stepping capabilities
- Mass balance checking and conservation

2.2.2 Physical Processes

- Conservative transport (advection-diffusion)
- Heat transport
- Multiple constituent tracking
- Wetting and drying capabilities

2.2.3 Integration Capabilities

- HEC-RAS 2D mesh compatibility
- ClearWater modules coupling (TSM, NSM)
- EFDC model comparison tools
- Custom boundary condition handling

2.3 Typical Applications

- **Pollutant Transport**: Tracking contaminant plumes in rivers
- **Heat Transport**: Thermal pollution studies
- **Nutrient Modeling**: Eutrophication assessment when coupled with NSM
- **Emergency Response**: Rapid assessment of spill impacts
- **Water Quality Planning**: Long-term water quality management

---

3. Installation and Setup

3.1 Prerequisites

Before installing ClearWater-Riverine, ensure you have:

1. **Python Environment**: Python 3.10 or higher
2. **Package Manager**: Miniconda or Anaconda Distribution
3. **Git** (optional): For cloning the repository

3.2 Step-by-Step Installation

3.2.1 Install Miniconda Download and install Miniconda from https://docs.conda.io/projects/miniconda/en/latest/

> **Important**: Install in your local user directory. Do NOT install for all users to avoid permission issues.

3.2.2 Obtain ClearWater-Riverine **Option A: Download ZIP** 1. Visit https://github.com/EcohydrologyTeam/ClearWater-riverine 2. Click the green "Code" button 3. Select "Download ZIP" 4. Extract to your desired location

Option B: Git Clone

```
git clone https://github.com/EcohydrologyTeam/ClearWater-riverine.git
cd ClearWater-riverine
```

3.2.3 Create Conda Environment Navigate to the ClearWater-riverine directory and create the environment:

```
conda env create -f environment.yml --solver=libmamba
```

If you experience plotting issues in Jupyter notebooks, use the working environment:

```
conda env create -f environment_working.yml --solver=libmamba
```

```
conda activate ClearWater-modules
```

3.2.4 Activate Environment

3.2.5 Add to Python Path Add ClearWater-riverine to your Python path:

```
conda develop '/path/to/ClearWater-riverine/src'
```

Replace `/path/to/ClearWater-riverine/src` with the actual path to your installation.

3.3 Verification

Test your installation by launching Python and importing the module:

```
import clearwater_riverine
print("ClearWater-Riverine installed successfully!")
```

3.4 Optional: Install Jupyter Extensions

For enhanced notebook experience:

```
pip install jupyterlab-variableInspector
```

---

4. Getting Started

4.1 Your First Model

This section walks you through creating your first ClearWater-Riverine model using the provided examples.

```
jupyter lab
```

4.1.1 Launch Jupyter Lab

4.1.2 Open Example Notebook Navigate to `examples/01_getting_started_riverine.ipynb` and open it.

4.1.3 Basic Model Structure Every ClearWater-Riverine model follows this basic structure:

```python
# 1. Import modules
import clearwater_riverine
from clearwater_riverine.io import hdf, inputs, outputs
from clearwater_riverine.transport import Transport

# 2. Load mesh
mesh = hdf.load_mesh('path/to/mesh.hdf')

# 3. Set initial conditions
initial_conditions = inputs.load_initial_conditions('initial_conditions.csv')

# 4. Set boundary conditions
boundary_conditions = inputs.load_boundary_conditions('boundary_conditions.csv')

# 5. Create transport model
transport = Transport(mesh)

# 6. Run simulation
results = transport.run(initial_conditions, boundary_conditions)

# 7. Save results
outputs.save_results(results, 'output_directory')
```

4.2 Understanding the Workflow

4.2.1 Model Preparation

1. **Mesh Setup**: Import computational mesh from HEC-RAS or create custom mesh

2. **Initial Conditions**: Define starting concentrations for all constituents
3. **Boundary Conditions**: Specify inflows, outflows, and constituent loadings
4. **Parameters**: Set transport parameters (diffusion coefficients, time steps)

4.2.2 Model Execution

1. **Initialization**: Create transport solver with mesh
2. **Time Stepping**: Advance solution through time
3. **Mass Balance**: Check conservation at each step
4. **Output**: Store results at specified intervals

4.2.3 Post-Processing

1. **Visualization**: Create plots and animations
2. **Analysis**: Extract time series and spatial distributions
3. **Validation**: Compare with observations or other models

4.3 Quick Start Checklist

- ☐ Environment activated (`conda activate ClearWater-modules`)
- ☐ Jupyter Lab launched
- ☐ Example notebook opened
- ☐ Test data files located
- ☐ First model run completed successfully

---

5. Model Components

5.1 Transport Module

The transport module (`clearwater_riverine.transport`) is the core component that handles advection and diffusion processes.

```
from clearwater_riverine.transport import Transport

# Initialize with mesh
transport = Transport(mesh, parameters)
```

5.1.1 Transport Class Key Methods: - `run()`: Execute full simulation - `step()`: Advance one time step - `set_boundary_conditions()`: Update boundary conditions - `get_mass_balance()`: Check conservation

5.1.2 Advection Process Advection transports constituents with the flow field:

- Uses flow velocities from hydrodynamic model
- Implements upwind finite volume scheme
- Handles varying flow directions and magnitudes

5.1.3 Diffusion Process Diffusion represents mixing processes:

- Turbulent diffusion from flow shear
- Dispersion from velocity variations
- User-specified diffusion coefficients

5.2 Mesh Module

The mesh module (`clearwater_riverine.mesh`) handles computational grid operations.

```
from clearwater_riverine.mesh import Mesh

# Load from HDF file
mesh = Mesh.from_hdf('model.hdf')

# Key properties
print(f"Number of cells: {mesh.n_cells}")
print(f"Number of faces: {mesh.n_faces}")
```

5.2.1 Mesh Class **Key Attributes:** - `cell_centers`: Cell centroid coordinates - `face_centers`: Face centroid coordinates
- `cell_volumes`: Volume of each cell - `face_areas`: Area of each face - `connectivity`: Cell-face connectivity

5.2.2 Grid Types **Unstructured Grids:** - Triangular and quadrilateral cells - Flexible boundary representation - Adaptive refinement capability

Structured Grids: - Regular rectangular cells - Simplified connectivity - Efficient for simple geometries

5.3 Input/Output Module

The I/O module (`clearwater_riverine.io`) handles data import and export.

```
from clearwater_riverine.io import hdf

# Load mesh from HDF5 file
mesh = hdf.load_mesh('model.hdf')

# Load results
results = hdf.load_results('output.hdf')
```

5.3.1 HDF Module

```
from clearwater_riverine.io import inputs
```

```
# Load initial conditions
ic = inputs.load_initial_conditions('initial.csv')

# Load boundary conditions
bc = inputs.load_boundary_conditions('boundary.csv')
```

5.3.2 Inputs Module

```
from clearwater_riverine.io import outputs

# Save results to HDF5
outputs.save_hdf(results, 'output.hdf')

# Export to CSV
outputs.save_csv(results, 'output.csv')
```

5.3.3 Outputs Module

5.4 Utilities Module

The utilities module (`clearwater_riverine.utilities`) provides helper functions.

```
from clearwater_riverine.utilities import (
    calculate_time_step,
    check_mass_balance,
    interpolate_values
)

# Calculate stable time step
dt = calculate_time_step(mesh, velocities, diffusion_coeff)

# Check mass conservation
mass_error = check_mass_balance(old_values, new_values, sources, sinks)
```

5.4.1 Common Utilities

---

6. Workflows and Tutorials

6.1 Basic Transport Simulation

This tutorial demonstrates a simple conservative transport simulation.

6.1.1 Problem Setup **Scenario**: Pollutant spill in a river channel - Domain: 1000m x 100m river reach - Flow: Steady downstream flow - Pollutant: Conservative tracer

6.1.2 Step-by-Step Procedure Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from clearwater_riverine import Transport
from clearwater_riverine.io import hdf, inputs, outputs
```

Step 2: Load Model Data

```
# Load computational mesh
mesh = hdf.load_mesh('examples/data/simple_channel.hdf')

# Load flow field
flow_data = hdf.load_flow_field('examples/data/flow_field.hdf')
```

Step 3: Set Initial Conditions

```
# Create initial concentration field (mg/L)
initial_conc = np.zeros(mesh.n_cells)

# Add point source at upstream location
source_cells = mesh.find_cells_near_point(x=100, y=50)
initial_conc[source_cells] = 100.0
```

Step 4: Set Boundary Conditions

```
# Define upstream boundary
upstream_bc = {
    'cells': mesh.get_boundary_cells('upstream'),
    'type': 'concentration',
    'values': 0.0  # Clean water inflow
}

# Define downstream boundary
downstream_bc = {
    'cells': mesh.get_boundary_cells('downstream'),
    'type': 'zero_gradient'
}

boundary_conditions = [upstream_bc, downstream_bc]
```

Step 5: Configure Transport Parameters

```
transport_params = {
    'diffusion_coefficient': 10.0,  # m²/s
    'time_step': 10.0,              # seconds
    'total_time': 3600.0,           # 1 hour
    'output_interval': 300.0        # 5 minutes
}
```

Step 6: Run Simulation

```
# Create transport solver
transport = Transport(mesh, transport_params)

# Run simulation
results = transport.run(
    initial_conditions=initial_conc,
    boundary_conditions=boundary_conditions,
    flow_field=flow_data
)
```

Step 7: Visualize Results

```
# Plot concentration at different times
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
times = [0, 300, 600, 1200]  # seconds

for i, t in enumerate(times):
    ax = axes[i//2, i%2]
    mesh.plot_contour(results[t], ax=ax, levels=20)
    ax.set_title(f'Concentration at t = {t/60:.0f} min')
    ax.set_xlabel('Distance (m)')
    ax.set_ylabel('Width (m)')

plt.tight_layout()
plt.show()
```

6.2 Heat Transport Simulation

This tutorial shows how to simulate heat transport in a river.

6.2.1 Problem Setup **Scenario**: Thermal discharge from power plant - Warm water discharge at 35°C - Ambient river temperature: 20°C - Surface heat exchange with atmosphere

```
# Heat-specific parameters
heat_params = {
    'diffusion_coefficient': 50.0,    # Higher for heat
    'surface_heat_exchange': True,
    'air_temperature': 25.0,          # °C
    'wind_speed': 3.0,                # m/s
    'solar_radiation': 500.0          # W/m²
}

# Temperature boundary condition
thermal_discharge = {
    'cells': discharge_cells,
    'type': 'temperature',
```

```
    'values': 35.0  # °C
}
```

6.2.2 Key Differences from Conservative Transport

6.3 Multi-Constituent Simulation

This tutorial demonstrates tracking multiple constituents simultaneously.

6.3.1 Problem Setup **Scenario**: Nutrient transport (nitrogen and phosphorus) - Point source discharge - Different decay rates - Interaction between constituents

```
# Define constituents
constituents = ['nitrogen', 'phosphorus', 'algae']

# Initial conditions for each constituent
initial_conditions = {
    'nitrogen': np.ones(mesh.n_cells) * 2.0,    # mg/L
    'phosphorus': np.ones(mesh.n_cells) * 0.1,  # mg/L
    'algae': np.ones(mesh.n_cells) * 5.0        #  g/L
}

# Run multi-constituent transport
results = {}
for constituent in constituents:
    transport = Transport(mesh, params[constituent])
    results[constituent] = transport.run(
        initial_conditions=initial_conditions[constituent],
        boundary_conditions=boundary_conditions[constituent]
    )
```

6.3.2 Implementation

---

7. Input File Formats

7.1 Mesh Files (HDF5 Format)

ClearWater-Riverine uses HDF5 format for mesh files, typically exported from HEC-RAS 2D.

7.1.1 Required Datasets Geometry Groups:

```
/Geometry/
   2D Flow Areas/
       Attributes/
       Cell Centers/
       Cell Points/
       Cells/
```

Face Points/

Key Arrays: - `Cell Centers`: Cell centroid coordinates (x, y) - `Cell Points`: Vertex coordinates for each cell - `Cells`: Cell connectivity information - `Face Points`: Face midpoint coordinates

7.1.2 Optional Datasets Flow Data: - `Velocity`: Flow velocities (u, v components) - `Water Surface Elevation`: Water levels - `Cell Volume`: Cell volumes (if variable)

7.2 Initial Conditions (CSV Format)

Initial conditions are specified in CSV format with the following structure:

7.2.1 File Format

```
CellID,Concentration,Temperature,Constituent2
1,0.0,20.0,1.5
2,0.0,20.1,1.4
3,0.0,20.0,1.6
...
```

7.2.2 Column Definitions

- `CellID`: Unique cell identifier (integer)
- `Concentration`: Initial concentration (mg/L or appropriate units)
- `Temperature`: Initial temperature (°C)
- Additional columns for other constituents

```python
# Create initial conditions programmatically
import pandas as pd

# Generate initial conditions
n_cells = mesh.n_cells
initial_data = {
    'CellID': range(1, n_cells + 1),
    'Concentration': np.zeros(n_cells),
    'Temperature': np.full(n_cells, 20.0)
}

# Add point source
source_cells = mesh.find_cells_near_point(x=500, y=50)
initial_data['Concentration'][source_cells] = 100.0

# Save to CSV
df = pd.DataFrame(initial_data)
df.to_csv('initial_conditions.csv', index=False)
```

7.2.3 Example File

7.3 Boundary Conditions (CSV Format)

Boundary conditions define how constituents enter or leave the model domain.

7.3.1 Time Series Format

```
DateTime,Flow,Concentration,Temperature
2023-01-01 00:00:00,100.5,2.5,15.2
2023-01-01 01:00:00,98.2,2.4,15.1
2023-01-01 02:00:00,95.8,2.3,15.0
...
```

7.3.2 Boundary Types Concentration Boundary: - Specifies constituent concentration at boundary - Used for inflow boundaries - Format: `concentration = value` (mg/L)

Flow Boundary: - Specifies mass flux across boundary - Used for point sources/sinks - Format: `mass_flux = value` (kg/s)

Zero Gradient: - Natural boundary condition - Used for outflow boundaries - No additional specification required

```
# Define multiple boundary conditions
boundary_conditions = {
    'upstream': {
        'file': 'upstream_bc.csv',
        'type': 'concentration',
        'cells': upstream_cells
    },
    'point_source': {
        'file': 'point_source_bc.csv',
        'type': 'mass_flux',
        'cells': source_cells
    },
    'downstream': {
        'type': 'zero_gradient',
        'cells': downstream_cells
    }
}
```

7.3.3 Multiple Boundaries

7.4 Configuration Files (YAML Format)

Model configuration can be specified using YAML files for complex setups.

```
# model_config.yml
model:
  name: "Ohio River E.coli Transport"
```

```yaml
  description: "E.coli transport simulation"

mesh:
  file: "data/ohio_river.hdf"

transport:
  constituents:
    - name: "ecoli"
      diffusion_coefficient: 10.0
      units: "CFU/100mL"

time:
  start: "2010-06-01 00:00:00"
  end: "2010-06-02 00:00:00"
  step: 30.0  # seconds
  output_interval: 300.0  # seconds

boundaries:
  upstream:
    type: "concentration"
    file: "data/upstream_bc.csv"
    cells: [1, 2, 3]

  point_source:
    type: "mass_flux"
    file: "data/point_source.csv"
    cells: [1250]

output:
  directory: "results/"
  format: ["hdf5", "csv"]
  variables: ["concentration", "mass_balance"]
```

7.4.1 Example Configuration

---

8. Output File Formats

8.1 Results Files (HDF5 Format)

ClearWater-Riverine saves results in HDF5 format for efficient storage and retrieval.

8.1.1 File Structure

```
results.hdf5/
   Metadata/
       Model_Info
       Time_Info
```

```
    Mesh_Info
  Results/
    Concentration/
      Time_00000
      Time_00001
      ...
    Velocity/
    Mass_Balance/
  Mesh/
    Cell_Centers
    Cell_Volumes
    Connectivity
```

```
import h5py
import numpy as np

# Open results file
with h5py.File('results.hdf5', 'r') as f:
    # Read time information
    times = f['Metadata/Time_Info/Times'][:]

    # Read concentration at specific time
    conc_t0 = f['Results/Concentration/Time_00000'][:]

    # Read all concentrations
    all_conc = []
    for i in range(len(times)):
        time_key = f'Time_{i:05d}'
        conc = f[f'Results/Concentration/{time_key}'][:]
        all_conc.append(conc)
```

8.1.2 Reading Results

8.2 Time Series Files (CSV Format)

Point-specific time series data is exported in CSV format.

8.2.1 Format Structure

```
DateTime,CellID,X,Y,Concentration,Temperature
2023-01-01 00:00:00,1250,1000.5,250.2,15.6,20.1
2023-01-01 00:05:00,1250,1000.5,250.2,18.2,20.2
2023-01-01 00:10:00,1250,1000.5,250.2,21.8,20.3
...
```

```
# Extract time series at monitoring points
monitoring_points = [
```

```
    {'name': 'Upstream', 'cell_id': 100},
    {'name': 'Midstream', 'cell_id': 500},
    {'name': 'Downstream', 'cell_id': 900}
]

time_series = {}
for point in monitoring_points:
    cell_id = point['cell_id']
    time_series[point['name']] = {
        'time': results['time'],
        'concentration': results['concentration'][:, cell_id],
        'temperature': results['temperature'][:, cell_id]
    }
```

8.2.2 Extracting Time Series

8.3 Visualization Files

8.3.1 VTK Export For advanced visualization in ParaView or VisIt:

```
from clearwater_riverine.io import vtk_export

# Export to VTK format
vtk_export.save_results(
    mesh=mesh,
    results=results,
    filename='results.vtk',
    variables=['concentration', 'temperature']
)
```

8.3.2 NetCDF Export For climate/oceanographic tools:

```
from clearwater_riverine.io import netcdf_export

# Export to NetCDF
netcdf_export.save_results(
    mesh=mesh,
    results=results,
    filename='results.nc',
    metadata={
        'title': 'ClearWater-Riverine Results',
        'institution': 'ERDC-EL',
        'source': 'ClearWater-Riverine v1.0'
    }
)
```

---

9. Model Parameters

9.1 Transport Parameters

9.1.1 Diffusion Coefficient The diffusion coefficient controls the rate of mixing and spreading.

Typical Values: - Rivers: 10-100 m^2/s - Estuaries: 50-500 m^2/s - Lakes: 1-50 m^2/s

Selection Guidelines:

```
# Calculate diffusion coefficient from flow characteristics
def estimate_diffusion(velocity, width, depth):
    """
    Estimate diffusion coefficient using Elder's formula
    """
    # Elder's coefficient (typically 0.23)
    beta = 0.23

    # Shear velocity
    u_star = np.sqrt(9.81 * depth * 0.001)  # Assuming slope = 0.001

    # Transverse diffusion
    D_y = beta * depth * u_star

    # Longitudinal diffusion
    D_x = 5.93 * depth * u_star

    return D_x, D_y
```

9.1.2 Time Step Time step selection affects stability and accuracy.

Stability Criteria:

```
def calculate_max_timestep(mesh, velocity, diffusion):
    """
    Calculate maximum stable time step
    """
    # Courant number criterion (advection)
    min_cell_size = np.min(mesh.cell_sizes)
    max_velocity = np.max(velocity)
    dt_courant = 0.5 * min_cell_size / max_velocity

    # Diffusion criterion
    dt_diffusion = 0.25 * min_cell_size**2 / diffusion

    # Take minimum
    dt_max = min(dt_courant, dt_diffusion)

    return dt_max
```

9.2 Numerical Parameters

```
solver_params = {
    'method': 'finite_volume',      # Discretization method
    'scheme': 'upwind',             # Advection scheme
    'limiter': 'minmod',            # Flux limiter
    'tolerance': 1e-6,              # Convergence tolerance
    'max_iterations': 1000          # Maximum iterations
}
```

9.2.1 Solver Settings

```
matrix_params = {
    'solver': 'gmres',              # Linear solver
    'preconditioner': 'ilu',        # Preconditioner
    'fill_factor': 2.0,             # ILU fill factor
    'drop_tolerance': 1e-4          # Drop tolerance
}
```

9.2.2 Matrix Solver Options

9.3 Physical Parameters

```
heat_params = {
    'thermal_diffusivity': 1.4e-7,  # m²/s
    'heat_exchange_coeff': 20.0,    # W/m²/K
    'solar_absorption': 0.7,        # Fraction
    'longwave_emissivity': 0.97,    # Fraction
    'wind_function': 'ryan_harleman' # Wind function type
}
```

9.3.1 Heat Transport Parameters

```
mass_params = {
    'schmidt_number': 600.0,        # For oxygen
    'reaeration_formula': 'owens',  # O'Connor-Dobbins, Owens, etc.
    'wind_reaeration': True,        # Include wind effects
    'temperature_correction': 1.024 # Arrhenius factor
}
```

9.3.2 Mass Transfer Parameters

---

10. Coupling with ClearWater Modules

10.1 Temperature Simulation Module (TSM)

The Temperature Simulation Module simulates heat transport and thermal processes.

```
from clearwater_modules import TSM
from clearwater_riverine import Transport

# Initialize models
transport = Transport(mesh)
tsm = TSM()

# Coupling parameters
coupling_params = {
    'time_step': 60.0,              # seconds
    'coupling_interval': 300.0,     # seconds
    'heat_exchange': True,
    'solar_radiation': True
}

# Meteorological data
met_data = {
    'air_temperature': 25.0,        # °C
    'wind_speed': 3.0,              # m/s
    'relative_humidity': 0.7,       # fraction
    'solar_radiation': 500.0,       # W/m²
    'cloud_cover': 0.3              # fraction
}
```

10.1.1 TSM Integration

```
# Main simulation loop
for time_step in range(n_steps):
    current_time = start_time + time_step * dt

    # Transport temperature
    temperature = transport.step(
        variable=temperature,
        time_step=dt,
        boundary_conditions=thermal_bc
    )

    # Apply heat exchange processes
    if time_step % coupling_interval == 0:
        # Surface heat exchange
        heat_flux = tsm.calculate_surface_heat_flux(
```

```
            temperature=temperature,
            meteorology=met_data[current_time]
        )

        # Update temperature
        temperature += heat_flux * coupling_interval / (
            mesh.cell_volumes * water_density * specific_heat
        )

    # Store results
    results['temperature'][time_step] = temperature.copy()
```

10.1.2 Coupled Simulation Loop

10.2 Nutrient Simulation Module (NSM)

The Nutrient Simulation Module simulates nutrient cycling and eutrophication processes.

```
from clearwater_modules import NSM

# Initialize NSM
nsm = NSM()

# Define constituents
constituents = [
    'dissolved_oxygen',
    'organic_nitrogen',
    'ammonia_nitrogen',
    'nitrate_nitrogen',
    'organic_phosphorus',
    'dissolved_phosphorus',
    'phytoplankton',
    'detritus'
]

# NSM parameters
nsm_params = {
    'maximum_growth_rate': 2.0,      # 1/day
    'half_saturation_N': 0.025,      # mg/L
    'half_saturation_P': 0.0025,     # mg/L
    'respiration_rate': 0.05,        # 1/day
    'mortality_rate': 0.1,           # 1/day
    'settling_velocity': 1.0,        # m/day
    'temperature_coefficient': 1.047
}
```

10.2.1 NSM Integration

```
# Initialize constituent concentrations
concentrations = {}
for constituent in constituents:
    concentrations[constituent] = initial_conditions[constituent].copy()

# Main simulation loop
for time_step in range(n_steps):
    current_time = start_time + time_step * dt

    # Transport all constituents
    for constituent in constituents:
        concentrations[constituent] = transport.step(
            variable=concentrations[constituent],
            time_step=dt,
            boundary_conditions=boundary_conditions[constituent]
        )

    # Apply biogeochemical processes
    if time_step % nsm_coupling_interval == 0:
        # Calculate reaction rates
        reaction_rates = nsm.calculate_rates(
            concentrations=concentrations,
            temperature=temperature,
            light=light_field
        )

        # Update concentrations
        for constituent in constituents:
            rate = reaction_rates[constituent]
            concentrations[constituent] += rate * nsm_dt

            # Apply settling for particulate matter
            if constituent in ['phytoplankton', 'detritus']:
                settling_loss = nsm.calculate_settling(
                    concentration=concentrations[constituent],
                    settling_velocity=nsm_params['settling_velocity'],
                    depth=mesh.cell_depths
                )
                concentrations[constituent] -= settling_loss

    # Store results
    for constituent in constituents:
        results[constituent][time_step] = concentrations[constituent].copy()
```

10.2.2 Coupled NSM Simulation

10.3 Advanced Coupling Techniques

```
def operator_splitting_step(transport, nsm, concentrations, dt):
    """
    Operator splitting approach for transport-reaction coupling
    """
    # Step 1: Transport (advection-diffusion)
    for constituent in concentrations:
        concentrations[constituent] = transport.step(
            concentrations[constituent], dt/2
        )

    # Step 2: Reaction (biogeochemistry)
    concentrations = nsm.react(concentrations, dt)

    # Step 3: Transport (second half)
    for constituent in concentrations:
        concentrations[constituent] = transport.step(
            concentrations[constituent], dt/2
        )

    return concentrations
```

10.3.1 Operator Splitting

```
def adaptive_coupling(transport, nsm, concentrations, dt_max):
    """
    Adaptive time stepping for coupled simulation
    """
    dt = dt_max
    error_tolerance = 0.01

    while True:
        # Trial step with full time step
        conc_full = operator_splitting_step(
            transport, nsm, concentrations.copy(), dt
        )

        # Two half steps
        conc_half1 = operator_splitting_step(
            transport, nsm, concentrations.copy(), dt/2
        )
        conc_half2 = operator_splitting_step(
            transport, nsm, conc_half1, dt/2
        )
```

```
        # Estimate error
        max_error = 0
        for constituent in concentrations:
            error = np.max(np.abs(conc_full[constituent] - conc_half2[constituent]))
            max_error = max(max_error, error)

        # Check convergence
        if max_error < error_tolerance:
            return conc_full, dt
        else:
            dt *= 0.8  # Reduce time step
```

10.3.2 Adaptive Time Stepping

---

11. Visualization and Post-Processing

11.1 Basic Plotting

```
import matplotlib.pyplot as plt
import numpy as np

def plot_concentration_contours(mesh, concentration, title="Concentration"):
    """
    Create contour plot of concentration field
    """
    fig, ax = plt.subplots(figsize=(12, 8))

    # Create contour plot
    levels = np.linspace(concentration.min(), concentration.max(), 20)
    contour = ax.tricontourf(
        mesh.cell_centers[:, 0],  # x coordinates
        mesh.cell_centers[:, 1],  # y coordinates
        concentration,
        levels=levels,
        cmap='viridis'
    )

    # Add colorbar
    cbar = plt.colorbar(contour, ax=ax)
    cbar.set_label('Concentration (mg/L)')

    # Format plot
    ax.set_xlabel('Distance (m)')
    ax.set_ylabel('Width (m)')
    ax.set_title(title)
    ax.axis('equal')
```

```
    return fig, ax
```

11.1.1 Contour Plots

```
def plot_time_series(times, concentrations, locations, labels=None):
    """
    Plot concentration time series at multiple locations
    """
    fig, ax = plt.subplots(figsize=(10, 6))

    if labels is None:
        labels = [f'Location {i+1}' for i in range(len(locations))]

    for i, (location, label) in enumerate(zip(locations, labels)):
        ax.plot(times, concentrations[:, location],
                label=label, linewidth=2)

    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('Concentration (mg/L)')
    ax.set_title('Concentration Time Series')
    ax.legend()
    ax.grid(True, alpha=0.3)

    return fig, ax
```

11.1.2 Time Series Plots

11.2 Animation Creation

```
import matplotlib.animation as animation

def create_concentration_animation(mesh, results, filename='animation.gif'):
    """
    Create animated GIF of concentration evolution
    """
    fig, ax = plt.subplots(figsize=(12, 8))

    # Initialize plot
    times = results['times']
    concentrations = results['concentration']

    # Set up contour levels
    vmin = np.min(concentrations)
    vmax = np.max(concentrations)
    levels = np.linspace(vmin, vmax, 20)
```

```
    def animate(frame):
        ax.clear()

        # Create contour plot for current time
        contour = ax.tricontourf(
            mesh.cell_centers[:, 0],
            mesh.cell_centers[:, 1],
            concentrations[frame],
            levels=levels,
            cmap='viridis',
            vmin=vmin,
            vmax=vmax
        )

        # Format plot
        ax.set_xlabel('Distance (m)')
        ax.set_ylabel('Width (m)')
        ax.set_title(f'Concentration at t = {times[frame]/3600:.1f} hours')
        ax.axis('equal')

        return contour.collections

    # Create animation
    anim = animation.FuncAnimation(
        fig, animate, frames=len(times),
        interval=200, blit=False
    )

    # Save animation
    anim.save(filename, writer='pillow', fps=5)
    plt.close()

    return anim
```

11.2.1 Concentration Animation

11.3 Advanced Visualization

```
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots

def create_interactive_plot(mesh, results):
    """
    Create interactive concentration plot using Plotly
    """
```

```
    # Create subplot with secondary y-axis
    fig = make_subplots(
        rows=2, cols=2,
        subplot_titles=('Concentration Field', 'Time Series',
                        'Mass Balance', 'Cross Section'),
        specs=[[{"type": "scatter"}, {"type": "scatter"}],
               [{"type": "scatter"}, {"type": "scatter"}]]
    )

    # Concentration contour
    fig.add_trace(
        go.Scatter(
            x=mesh.cell_centers[:, 0],
            y=mesh.cell_centers[:, 1],
            mode='markers',
            marker=dict(
                size=8,
                color=results['concentration'][-1],
                colorscale='Viridis',
                showscale=True
            ),
            name='Final Concentration'
        ),
        row=1, col=1
    )

    # Time series
    times = results['times'] / 3600  # Convert to hours
    monitoring_cells = [100, 500, 900]

    for i, cell in enumerate(monitoring_cells):
        fig.add_trace(
            go.Scatter(
                x=times,
                y=results['concentration'][:, cell],
                mode='lines',
                name=f'Cell {cell}',
                line=dict(width=2)
            ),
            row=1, col=2
        )

    # Mass balance
    mass_balance = results['mass_balance']
    fig.add_trace(
        go.Scatter(
            x=times,
```

```
            y=mass_balance,
            mode='lines',
            name='Mass Balance Error',
            line=dict(color='red', width=2)
        ),
        row=2, col=1
    )

    # Update layout
    fig.update_layout(
        height=800,
        title_text='ClearWater-Riverine Results Dashboard',
        showlegend=True
    )

    return fig
```

11.3.1 Interactive Plots with Plotly

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def plot_3d_concentration(mesh, concentration, elevation_scale=100):
    """
    Create 3D visualization with concentration as height
    """
    fig = plt.figure(figsize=(12, 9))
    ax = fig.add_subplot(111, projection='3d')

    # Use concentration as elevation
    x = mesh.cell_centers[:, 0]
    y = mesh.cell_centers[:, 1]
    z = concentration * elevation_scale

    # Create 3D surface
    ax.scatter(x, y, z, c=concentration, cmap='viridis', s=50)

    # Format plot
    ax.set_xlabel('Distance (m)')
    ax.set_ylabel('Width (m)')
    ax.set_zlabel(f'Concentration × {elevation_scale}')
    ax.set_title('3D Concentration Distribution')

    return fig, ax
```

11.3.2 3D Visualization

11.4 Statistical Analysis

```
def calculate_spatial_stats(mesh, concentration):
    """
    Calculate spatial statistics of concentration field
    """
    # Basic statistics
    stats = {
        'mean': np.mean(concentration),
        'std': np.std(concentration),
        'min': np.min(concentration),
        'max': np.max(concentration),
        'median': np.median(concentration)
    }

    # Center of mass
    total_mass = np.sum(concentration * mesh.cell_volumes)
    if total_mass > 0:
        x_center = np.sum(concentration * mesh.cell_volumes *
                          mesh.cell_centers[:, 0]) / total_mass
        y_center = np.sum(concentration * mesh.cell_volumes *
                          mesh.cell_centers[:, 1]) / total_mass
        stats['center_of_mass'] = (x_center, y_center)

    # Spatial moments
    if total_mass > 0:
        x_var = np.sum(concentration * mesh.cell_volumes *
                       (mesh.cell_centers[:, 0] - x_center)**2) / total_mass
        y_var = np.sum(concentration * mesh.cell_volumes *
                       (mesh.cell_centers[:, 1] - y_center)**2) / total_mass
        stats['spatial_variance'] = (x_var, y_var)

    return stats
```

11.4.1 Spatial Statistics

```
def calculate_performance_metrics(observed, simulated):
    """
    Calculate model performance metrics
    """
    # Remove NaN values
    mask = ~(np.isnan(observed) | np.isnan(simulated))
    obs = observed[mask]
    sim = simulated[mask]

    if len(obs) == 0:
```

```
        return {}

    # Mean error
    ME = np.mean(sim - obs)

    # Mean absolute error
    MAE = np.mean(np.abs(sim - obs))

    # Root mean square error
    RMSE = np.sqrt(np.mean((sim - obs)**2))

    # Nash-Sutcliffe efficiency
    NSE = 1 - np.sum((obs - sim)**2) / np.sum((obs - np.mean(obs))**2)

    # Correlation coefficient
    correlation = np.corrcoef(obs, sim)[0, 1]

    # Percent bias
    PBIAS = 100 * np.sum(sim - obs) / np.sum(obs)

    metrics = {
        'ME': ME,
        'MAE': MAE,
        'RMSE': RMSE,
        'NSE': NSE,
        'R': correlation,
        'PBIAS': PBIAS,
        'n_points': len(obs)
    }

    return metrics
```

11.4.2 Model Performance Metrics

---

12. Troubleshooting

12.1 Common Installation Issues

12.1.1 Conda Environment Problems **Problem**: Environment creation fails

```
CondaError: Cannot create environment
```

Solution:

```
# Clean conda cache
conda clean --all

# Update conda
```

```
conda update conda

# Try creating environment with specific solver
conda env create -f environment.yml --solver=libmamba
```

Problem: Module import errors

```
ModuleNotFoundError: No module named 'clearwater_riverine'
```

Solution:

```
# Verify path is added
conda develop --help

# Add path manually
conda develop /full/path/to/ClearWater-riverine/src

# Verify installation
python -c "import clearwater_riverine; print('Success!')"
```

12.1.2 Jupyter Notebook Issues

Problem: Plots not displaying in notebooks

Solutions: 1. Use working environment: `bash conda env create -f environment_working.yml`

2. Enable matplotlib backend:

```
%matplotlib inline
import matplotlib.pyplot as plt
```

3. Install missing extensions:

```
jupyter labextension install @jupyter-widgets/jupyterlab-manager
```

12.2 Model Setup Issues

12.2.1 Mesh Loading Problems

Problem: HDF5 file cannot be read

```
KeyError: 'Geometry/2D Flow Areas'
```

Diagnosis:

```
import h5py

# Inspect HDF5 file structure
def inspect_hdf5(filename):
    with h5py.File(filename, 'r') as f:
        def print_structure(name, obj):
            print(name)
        f.visititems(print_structure)

inspect_hdf5('your_mesh.hdf')
```

Solutions: 1. Verify file was exported from HEC-RAS 2D 2. Check for required datasets 3. Use alternative mesh loading: `python from clearwater_riverine.io import inputs mesh = inputs.load_custom_mesh('your_mesh.hdf')`

12.2.2 Boundary Condition Errors

Problem: Boundary cells not found

```
ValueError: Boundary cells [1, 2, 3] not found in mesh
```

Solution:

```
# Find actual boundary cells
boundary_cells = mesh.find_boundary_cells('upstream')
print(f"Available boundary cells: {boundary_cells}")

# Update boundary conditions
bc['cells'] = boundary_cells
```

12.3 Runtime Issues

12.3.1 Numerical Instability

Problem: Solution becomes unstable (NaN values)

Symptoms: - Concentrations become negative or very large - NaN values appear in results - Mass balance errors grow rapidly

Diagnosis:

```
# Check time step stability
dt_max = transport.calculate_max_timestep()
print(f"Maximum stable time step: {dt_max:.2f} seconds")
print(f"Current time step: {transport.dt:.2f} seconds")

# Check mass balance
mass_error = transport.check_mass_balance()
print(f"Mass balance error: {mass_error:.6f}")
```

Solutions: 1. Reduce time step: `python transport.dt = dt_max * 0.5`

2. Lower diffusion coefficient:

   ```
   transport.diffusion_coefficient *= 0.5
   ```

3. Check boundary conditions for unrealistic values

12.3.2 Slow Performance

Problem: Simulation runs very slowly

Diagnosis:

```
import time
import cProfile

# Profile the simulation
cProfile.run('transport.run()', 'profile_results')
```

```
# Analyze results
import pstats
stats = pstats.Stats('profile_results')
stats.sort_stats('cumtime').print_stats(20)
```

Solutions: 1. Increase time step (if stable): `python transport.dt = min(dt_max * 0.8, transport.dt * 2)`

2. Reduce output frequency:

```
transport.output_interval *= 2
```

3. Use sparse matrix optimizations:

```
transport.use_sparse_matrices = True
```

12.4 Memory Issues

12.4.1 Out of Memory Errors **Problem**: Simulation crashes due to insufficient memory

Symptoms:

```
MemoryError: Unable to allocate array
```

Solutions: 1. Reduce output frequency: `python # Store fewer time steps transport.output_inter = 3600 # 1 hour instead of 5 minutes`

2. Use data compression:

```
# Enable HDF5 compression
transport.output_compression = True
```

3. Process results in chunks:

```
# Load results incrementally
for time_step in range(0, n_steps, chunk_size):
    results_chunk = transport.load_results_range(
        time_step, time_step + chunk_size
    )
    process_chunk(results_chunk)
```

---

13. Case Studies

13.1 Ohio River E. Coli Transport

This case study demonstrates modeling bacterial transport in a large river system.

13.1.1 Background **Location**: Ohio River between Louisville, KY and Cincinnati, OH **Scenario**: Combined sewer overflow event introduces E. coli **Objective**: Predict downstream transport and assess public health risk

13.1.2 Model Setup **Domain Characteristics**: - Length: 50 km river reach - Width: 200-800 m (variable) - Depth: 3-15 m (variable) - Cells: 12,500 computational cells

Flow Conditions: - Average discharge: 2,800 m^3/s - Flow velocity: 0.5-1.2 m/s - Reynolds number: ~10 (turbulent)

```
# Load Ohio River mesh
mesh = hdf.load_mesh('data/ohio_river/OhioRiver_m.p22.hdf')

# Set up E. coli transport
ecoli_params = {
    'diffusion_coefficient': 25.0,  # m²/s
    'decay_rate': 0.5,              # 1/day (T90 = 3.3 hours)
    'time_step': 30.0,              # seconds
    'simulation_time': 86400.0      # 24 hours
}

# Point source at Covington, KY
source_location = {'x': 1000.0, 'y': 500.0}  # UTM coordinates
source_cells = mesh.find_cells_near_point(**source_location)

# Initial conditions (CFU/100mL)
initial_ecoli = np.zeros(mesh.n_cells)
initial_ecoli[source_cells] = 10000.0  # High initial concentration

# Boundary conditions
boundary_conditions = {
    'upstream': {
        'type': 'concentration',
        'value': 0.0,  # Clean water
        'cells': mesh.get_boundary_cells('upstream')
    },
    'downstream': {
        'type': 'zero_gradient',
        'cells': mesh.get_boundary_cells('downstream')
    }
}

# Run simulation
transport = Transport(mesh, ecoli_params)
results = transport.run(
    initial_conditions=initial_ecoli,
    boundary_conditions=boundary_conditions
)
```

13.1.3 Implementation

13.1.4 Results Analysis **Plume Evolution**: - Peak concentration moves downstream at ~0.8 m/s - Lateral spreading occurs due to transverse mixing - Concentrations decrease due to dilution and decay

Travel Times: - 10 km downstream: 3.5 hours - 25 km downstream: 8.7 hours - 50 km downstream: 17.4 hours

Public Health Assessment:

```
# EPA recreation water quality standard
epa_standard = 126  # CFU/100mL (single sample maximum)

# Find exceedance areas
exceedance_mask = results['concentration'] > epa_standard
exceedance_area = np.sum(exceedance_mask * mesh.cell_areas, axis=1)

# Calculate duration of exceedance
times = results['times'] / 3600  # Convert to hours
duration_hours = np.sum(exceedance_mask, axis=0) * (times[1] - times[0])

print(f"Maximum exceedance area: {np.max(exceedance_area):.0f} m²")
print(f"Areas with >6 hour exceedance: {np.sum(duration_hours > 6)}")
```

13.2 Thermal Discharge Modeling

This case study examines thermal pollution from a power plant.

13.2.1 Background **Location**: Sumwere Creek near power plant **Scenario**: Cooling water discharge at elevated temperature **Objective**: Assess thermal impact on aquatic habitat

```

# Heat transport parameters
heat_params = {
    'thermal_diffusivity': 1.4e-7,  # m²/s
    'surface_heat_exchange': True,
    'meteorological_forcing': True
}

# Discharge conditions
discharge_temp = 35.0  # °C
discharge_flow = 5.0   # m³/s
ambient_temp = 20.0    # °C

# Temperature-dependent density effects
def calculate_buoyancy_effects(temperature, reference_temp=20.0):
    """Calculate density-driven flows"""
    beta = 2.1e-4  # Thermal expansion coefficient (1/°C)
    density_diff = beta * (temperature - reference_temp)
    return density_diff
```

13.2.2 Model Configuration

13.2.3 Results **Temperature Distribution**: - Thermal plume extends 500m downstream - Surface temperatures elevated by 3-5°C near discharge - Stratification occurs in deeper areas

Ecological Impact:

```
# Calculate thermal habitat suitability
def assess_thermal_habitat(temperature):
    """Assess habitat quality based on temperature"""
    optimal_range = (18, 24)  # °C for cold-water species

    suitable_mask = (temperature >= optimal_range[0]) & \
                    (temperature <= optimal_range[1])

    habitat_area = np.sum(suitable_mask * mesh.cell_areas)
    return habitat_area

# Calculate habitat loss
baseline_habitat = assess_thermal_habitat(ambient_temp * np.ones(mesh.n_cells))
impacted_habitat = [assess_thermal_habitat(temp) for temp in results['temperature']]

habitat_loss = (baseline_habitat - np.array(impacted_habitat)) / baseline_habitat * 100
print(f"Maximum habitat loss: {np.max(habitat_loss):.1f}%")
```

13.3 Nutrient Transport and Eutrophication

This case study demonstrates coupled transport-reaction modeling for nutrient management.

13.3.1 Background **Location**: Agricultural watershed tributary **Scenario**: Spring runoff carries agricultural nutrients **Objective**: Predict algal blooms and oxygen depletion

```
# Define nutrient constituents
constituents = [
    'dissolved_oxygen',      # mg/L
    'organic_nitrogen',      # mg N/L
    'ammonia_nitrogen',      # mg N/L
    'nitrate_nitrogen',      # mg N/L
    'dissolved_phosphorus',  # mg P/L
    'phytoplankton',         #  g Chl-a/L
    'detritus'               # mg/L
]

# Agricultural runoff boundary conditions
runoff_concentrations = {
    'organic_nitrogen': 5.0,
    'nitrate_nitrogen': 15.0,
```

```
    'dissolved_phosphorus': 2.0,
    'dissolved_oxygen': 8.0,
    'phytoplankton': 10.0,
    'detritus': 20.0
}
```

13.3.2 Multi-Constituent Setup

```
from clearwater_modules import NSM

# Initialize NSM with site-specific parameters
nsm = NSM()
nsm.set_parameters({
    'maximum_growth_rate': 1.5,      # 1/day (spring conditions)
    'half_saturation_N': 0.025,      # mg N/L
    'half_saturation_P': 0.0025,     # mg P/L
    'optimal_temperature': 20.0,     # °C
    'respiration_rate': 0.08,        # 1/day
    'mortality_rate': 0.12,          # 1/day
    'settling_velocity': 0.5,        # m/day
    'reaeration_rate': 2.0           # 1/day
})

# Coupled simulation
for time_step in range(n_steps):
    # Transport step
    for constituent in constituents:
        concentrations[constituent] = transport.step(
            concentrations[constituent], dt
        )

    # Biogeochemical reactions
    if time_step % nsm_interval == 0:
        reaction_rates = nsm.calculate_rates(
            concentrations, temperature, light_field
        )

        for constituent in constituents:
            concentrations[constituent] += reaction_rates[constituent] * nsm_dt
```

13.3.3 NSM Coupling

```
# Calculate eutrophication indicators
def assess_eutrophication(results):
    """Calculate eutrophication metrics"""
    metrics = {}
```

```
    # Chlorophyll-a (algal biomass)
    chl_a = results['phytoplankton']
    metrics['max_chlorophyll'] = np.max(chl_a)
    metrics['avg_chlorophyll'] = np.mean(chl_a)

    # Trophic state classification
    if metrics['avg_chlorophyll'] < 2.6:
        metrics['trophic_state'] = 'oligotrophic'
    elif metrics['avg_chlorophyll'] < 20:
        metrics['trophic_state'] = 'mesotrophic'
    else:
        metrics['trophic_state'] = 'eutrophic'

    # Oxygen depletion events
    do_min = np.min(results['dissolved_oxygen'])
    metrics['min_dissolved_oxygen'] = do_min
    metrics['hypoxic_risk'] = 'high' if do_min < 4.0 else 'low'

    # Nutrient limitation
    N_P_ratio = np.mean(results['nitrate_nitrogen']) / np.mean(results['dissolved_phosphorus'])
    if N_P_ratio > 16:
        metrics['limiting_nutrient'] = 'phosphorus'
    else:
        metrics['limiting_nutrient'] = 'nitrogen'

    return metrics

# Assess results
eutrophication_status = assess_eutrophication(results)
print(f"Trophic state: {eutrophication_status['trophic_state']}")
print(f"Limiting nutrient: {eutrophication_status['limiting_nutrient']}")
print(f"Hypoxic risk: {eutrophication_status['hypoxic_risk']}")
```

13.3.4 Eutrophication Assessment

---

14. Frequently Asked Questions

14.1 General Questions

Q: What types of water bodies can ClearWater-Riverine model?

A: ClearWater-Riverine is designed for riverine systems including: - Rivers and streams - Estuaries (well-mixed conditions) - Shallow lakes and reservoirs - Floodplains and wetlands - Coastal areas (2D, depth-averaged)

It assumes vertical homogeneity, so it's not suitable for stratified systems.

Q: How does ClearWater-Riverine compare to other water quality models?

A: Comparisons with common models:

Feature	ClearWater-Riverine	EFDC	CE-QUAL-W2	WASP
Dimensions	2D horizontal	3D	2D longitudinal-vertical	Variable
Grid type	Unstructured	Structured/Unstructured	Structured	Variable
Language	Python	Fortran	Fortran	Fortran
User interface	Jupyter notebooks	GUI/command line	GUI	GUI
Open source	Yes	Yes	No	No
Modern libraries	Yes	No	No	No

14.2 Technical Questions

Q: What are the model's computational requirements?

A: Requirements scale with problem size:

Domain Size	Cells	RAM	Runtime (24h sim)
Small	<1,000	2 GB	<1 minute
Medium	1,000-10,000	4 GB	1-10 minutes
Large	10,000-100,000	8 GB	10-60 minutes
Very Large	>100,000	16+ GB	1+ hours

Q: How do I choose appropriate time steps?

A: Time step selection depends on several factors:

```
# Stability-limited time step
dt_stability = 0.5 * min_cell_size / max_velocity

# Accuracy-limited time step
dt_accuracy = min_cell_size / (10 * max_velocity)

# Process-limited time step (for reaction coupling)
dt_process = 1.0 / max_reaction_rate

# Use the minimum
dt = min(dt_stability, dt_accuracy, dt_process)
```

Q: Can I use my own mesh format?

A: ClearWater-Riverine primarily supports HEC-RAS HDF5 format, but you can create custom loaders:

```
from clearwater_riverine.mesh import Mesh

def load_custom_mesh(filename):
    """Load mesh from custom format"""
    # Read your mesh format
```

```
    cell_centers, connectivity, volumes = read_custom_format(filename)

    # Create ClearWater mesh object
    mesh = Mesh(
        cell_centers=cell_centers,
        connectivity=connectivity,
        cell_volumes=volumes
    )

    return mesh
```

14.3 Modeling Questions

Q: How do I handle wetting and drying?

A: ClearWater-Riverine includes wetting/drying capabilities:

```
# Enable wetting/drying
transport.enable_wetting_drying = True
transport.minimum_depth = 0.01  # m

# Cells below minimum depth are deactivated
# Mass is conserved during wetting/drying transitions
```

Q: Can I model multiple species simultaneously?

A: Yes, you can track multiple constituents:

```
# Define multiple species
species = ['conservative_tracer', 'bacteria', 'temperature']

# Run transport for each species
results = {}
for species_name in species:
    transport_params = get_params(species_name)
    transport = Transport(mesh, transport_params)
    results[species_name] = transport.run(
        initial_conditions[species_name],
        boundary_conditions[species_name]
    )
```

Q: How do I validate my model?

A: Model validation should include:

1. **Mass balance checks**:

   ```
   mass_error = transport.check_mass_balance()
   assert abs(mass_error) < 1e-6, "Mass balance error too large"
   ```

2. **Comparison with analytical solutions**:

```
# For simple cases, compare with known solutions
analytical_solution = gaussian_plume(x, y, t, source_strength)
rmse = np.sqrt(np.mean((simulated - analytical_solution)**2))
```

3. **Comparison with observations**:

```
metrics = calculate_performance_metrics(observed, simulated)
print(f"Nash-Sutcliffe Efficiency: {metrics['NSE']:.3f}")
```

4. **Sensitivity analysis**:

```
# Test parameter sensitivity
for param_value in [0.5, 1.0, 2.0]:
    transport.diffusion_coefficient = base_value * param_value
    results = transport.run(initial_conditions, boundary_conditions)
    analyze_sensitivity(results, param_value)
```

14.4 Troubleshooting Questions

Q: Why am I getting negative concentrations?

A: Negative concentrations usually indicate numerical instability:

1. **Reduce time step**:

```
transport.dt *= 0.5
```

2. **Check boundary conditions** for unrealistic values

3. **Enable positivity constraints**:

```
transport.enforce_positivity = True
```

Q: My simulation is very slow. How can I speed it up?

A: Several optimization strategies:

1. **Increase time step** (if stable):

```
dt_max = transport.calculate_max_timestep()
transport.dt = dt_max * 0.8
```

2. **Reduce output frequency**:

```
transport.output_interval = 3600  # Output every hour instead of every minute
```

3. **Use sparse matrices**:

```
transport.use_sparse_solver = True
```

4. **Parallel processing** (if available):

```
transport.n_processors = 4
```

Q: How do I handle missing data in boundary conditions?

A: Several approaches for missing data:

```
# Linear interpolation
boundary_data = pd.read_csv('boundary_conditions.csv')
boundary_data.interpolate(method='linear', inplace=True)

# Forward fill for short gaps
boundary_data.fillna(method='ffill', limit=3, inplace=True)

# Use default values for remaining gaps
boundary_data.fillna(default_value, inplace=True)

# Or use more sophisticated interpolation
from scipy import interpolate
f = interpolate.interp1d(times[~np.isnan(values)], values[~np.isnan(values)],
                         kind='linear', fill_value='extrapolate')
filled_values = f(times)
```

---

15. References

15.1 Scientific References

1. **Fischer, H.B., List, E.J., Koh, R.C.Y., Imberger, J., & Brooks, N.H.** (1979). *Mixing in Inland and Coastal Waters*. Academic Press.
2. **Rutherford, J.C.** (1994). *River Mixing*. John Wiley & Sons.
3. **Martin, J.L., & McCutcheon, S.C.** (1999). *Hydrodynamics and Transport for Water Quality Modeling*. Lewis Publishers.
4. **Chapra, S.C.** (2008). *Surface Water-Quality Modeling*. Waveland Press.
5. **Elder, J.W.** (1959). The dispersion of marked fluid in turbulent shear flow. *Journal of Fluid Mechanics*, 5(4), 544-560.

15.2 Model Documentation

6. **U.S. Army Corps of Engineers** (2016). *HEC-RAS River Analysis System 2D Modeling User's Manual*. Hydrologic Engineering Center.
7. **Hamrick, J.M.** (2007). *The Environmental Fluid Dynamics Code User Manual*. Tetra Tech, Inc.
8. **Cole, T.M., & Wells, S.A.** (2017). *CE-QUAL-W2: A Two-Dimensional, Laterally Averaged, Hydrodynamic and Water Quality Model, Version 4.2*. Department of Civil and Environmental Engineering, Portland State University.

15.3 Numerical Methods

9. **LeVeque, R.J.** (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press.

10. **Ferziger, J.H., & Perić, M.** (2002). *Computational Methods for Fluid Dynamics*. Springer-Verlag.

11. **Patankar, S.V.** (1980). *Numerical Heat Transfer and Fluid Flow*. Taylor & Francis.

15.4 Water Quality Processes

12. **Thomann, R.V., & Mueller, J.A.** (1987). *Principles of Surface Water Quality Modeling and Control*. Harper Collins.

13. **Schnoor, J.L.** (1996). *Environmental Modeling: Fate and Transport of Pollutants in Water, Air, and Soil*. John Wiley & Sons.

14. **Lung, W.S.** (2001). *Water Quality Modeling for Wasteload Allocations and TMDLs*. John Wiley & Sons.

15.5 Python and Scientific Computing

15. **McKinney, W.** (2018). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.

16. **VanderPlas, J.** (2016). *Python Data Science Handbook*. O'Reilly Media.

17. **Harris, C.R., et al.** (2020). Array programming with NumPy. *Nature*, 585, 357-362.

15.6 Software Documentation

18. **NumPy Documentation** (2024). Available at: https://numpy.org/doc/

19. **SciPy Documentation** (2024). Available at: https://docs.scipy.org/

20. **Matplotlib Documentation** (2024). Available at: https://matplotlib.org/

21. **Pandas Documentation** (2024). Available at: https://pandas.pydata.org/docs/

22. **Jupyter Documentation** (2024). Available at: https://jupyter.org/documentation

15.7 Regulatory and Standards

23. **U.S. Environmental Protection Agency** (2015). *Water Quality Standards Handbook*. EPA Office of Water.

24. **World Health Organization** (2017). *Guidelines for Drinking-water Quality: Fourth Edition Incorporating the First Addendum*. WHO Press.

Appendices

Appendix A: Parameter Tables

[Detailed parameter tables would be included here]

Appendix B: Example Input Files

[Complete example input files would be provided here]

Appendix C: API Reference

[Quick reference to key functions and classes]

Appendix D: Conversion Factors

[Unit conversion tables and constants]

Document Information - **Version**: 1.0 - **Date**: August 2025 - **Authors**: U.S. Army Engineer Research and Development Center (ERDC) - **Contact**: Environmental Laboratory (EL) - **License**: [Specify license]

This manual is a comprehensive guide to using ClearWater-Riverine for water quality modeling. For the most current information, please refer to the online documentation and GitHub repository.